FECAP — FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO

Análise e Desenvolvimento de Sistemas — 2º semestre de 2026

Matheus Gajewski de Melo — 26029468

Mel Oliva Motta — 26029351

Rayna Guimarães Froes — 26029441

RELATÓRIO DE ESTRUTURA DE DADOS

ClassSync — Plataforma de Acompanhamento Escolar

Projeto Interdisciplinar — Entrega 2

Grupo 17

Professor: Eduardo Savino

São Paulo

2026

1. Introdução

O ClassSync aplica estruturas de dados em cinco pontos do sistema para reduzir acessos ao banco e evitar percursos repetidos sobre as listas. Este relatório mostra cada um deles com o problema, a estrutura escolhida, o trecho de código e o ganho obtido.

O ClassSync é uma plataforma web de acompanhamento escolar bimestral. O professor registra a descrição, a média e as tags de cada aluno; a administração revisa e publica; e o responsável consulta o relatório publicado. O sistema usa React no frontend, Node.js com Express no backend e MySQL como banco de dados.

Na Entrega 1, a disciplina trabalhou a paginação de alunos com listas em uma aplicação separada, em C#. Nesta entrega, os mesmos conceitos aparecem aplicados no sistema real, em JavaScript.

2. Conceitos utilizados

A escolha de cada estrutura foi feita pelo custo das operações mais usadas em cada caso, medido pela notação O (Big O). Ela descreve como o trabalho cresce conforme cresce o volume de dados (n).

Notação	Significado	Exemplo
O(1)	custo constante, independente do tamanho	consultar uma chave em um Map
O(n)	cresce na mesma proporção dos dados	percorrer uma lista inteira uma vez
O(n · m)	para cada item de uma lista, percorre outra	comparar cada registro com todos os de outra lista
O(n²)	para cada item, percorre a mesma lista de novo	verificar repetição comparando todos com todos

As estruturas usadas no projeto são nativas do JavaScript:

- **Array (lista):** coleção ordenada por posição. Percorrer e fatiar são operações simples, mas procurar um item exige percorrer a lista, em O(n).
- **Map (tabela hash):** coleção de pares chave → valor. Inserir e consultar por chave custam O(1) em média, o que o torna adequado para agrupar e eliminar repetições.

- **Set (conjunto):** coleção de valores sem repetição. Adicionar e verificar se um valor existe custam O(1) em média.

3. Aplicações no projeto

O maior ganho está no backend, onde as tags de todos os acompanhamentos passaram a ser buscadas em uma única consulta. Os outros casos usam Map e Set para agrupar e eliminar repetições em um só percurso.

#	Onde	Estrutura	Custo sem a técnica	Custo com a técnica
3.1	Tags dos acompanhamentos (backend)	Consulta em lote + Map	n + 1 consultas ao banco	2 consultas ao banco
3.2	Tags sem repetição e contagem de turmas	Set	$O(n^2)$	O(n)
3.3	Opções dos filtros da lista	Map	$O(n^2)$	O(n)
3.4	Progresso por turma e disciplina	Map com chave composta	um percurso por grupo	um único percurso

3.1 Tags de todos os acompanhamentos em uma única consulta

Problema. Cada acompanhamento pode ter várias tags, guardadas na tabela `acompanhamento_tag`. Buscar as tags de cada registro separadamente geraria uma consulta por acompanhamento, além da consulta da lista. Esse padrão é conhecido como problema N+1: com 100 acompanhamentos, seriam 101 idas ao banco.

Solução. O repositório monta um array com os ids de todos os acompanhamentos da lista e busca as tags de todos eles de uma vez, com `IN (?)`. Depois, agrupa as tags em um Map e as distribui entre os registros.

```
// Backend/src/repositories/AcompanhamentoRepository.js
async #anexarTags(lista) {
  if (lista.length === 0) return;
  const ids = lista.map((a) => a.id);
  const [rows] = await pool.query(
    `SELECT atg.id_acompanhamento, tg.id_tag, tg.nome
       FROM acompanhamento_tag atg
       JOIN tag tg ON tg.id_tag = atg.id_tag
      WHERE atg.id_acompanhamento IN (?)`,
    [ids]
  );

  // Agrupa as tags por acompanhamento: um único percurso sobre as linhas
  const tagsPorAcompanhamento = new Map();
  for (const r of rows) {
    if (!tagsPorAcompanhamento.has(r.id_acompanhamento)) {
      tagsPorAcompanhamento.set(r.id_acompanhamento, []);
    }
    tagsPorAcompanhamento
      .get(r.id_acompanhamento)
      .push({ id: r.id_tag, nome: r.nome });
  }

  // Cada acompanhamento pega as suas tags direto pela chave
  for (const acompanhamento of lista) {
    acompanhamento.tags = tagsPorAcompanhamento.get(acompanhamento.id) || [];
  }
}
```

Ganho. A listagem passa a fazer no máximo 2 consultas, qualquer que seja o número de acompanhamentos. Como cada consulta envolve rede e processamento no MySQL, essa é a otimização de maior impacto no tempo de resposta. Na distribuição em memória, as tags são agrupadas em um Map com o id do acompanhamento como chave. Assim, cada linha de tag é visitada uma vez e cada acompanhamento encontra as suas tags em O(1), com custo total O(n + t) para n acompanhamentos e t tags. Com 100 acompanhamentos e 300 tags, são cerca de 400 operações, contra 30.000 comparações da versão anterior, que usava `filter` dentro de um `for`.

3.2 Set para eliminar repetições

Problema. O professor envia as tags como uma lista de ids, que pode chegar com valores repetidos, como `[1, 1, 3]`. Verificar repetição comparando cada item com todos os outros custaria O(n²).

Solução. O serviço converte a lista em um Set, que descarta os repetidos ao inserir, e volta para array com o operador spread.

```
// Backend/src/services/AcompanhamentoService.js (#validarTags)
const semRepetir = [...new Set(tags)]; // [1, 1, 3] → [1, 3]
```

A mesma ideia conta as turmas distintas no painel inicial do professor:

```
// Frontend/src/pages/DashboardProfessor/DashboardProfessor.jsx
const turmasComRegistros = new Set(registros.map((r) => r.turma.id)).size;
```

Ganho. Cada inserção no Set custa O(1) em média, então a lista inteira é tratada em O(n). A lista sem repetição também evita gravar a mesma tag duas vezes na tabela `acompanhamento_tag`.

3.3 Map para montar as opções dos filtros

Problema. A tela de acompanhamentos oferece filtros por turma, disciplina e bimestre, com opções tiradas dos próprios registros. Vários registros têm a mesma turma, e cada opção deve aparecer uma vez só. Procurar em um array se a turma já foi incluída, a cada registro, custaria O(n²).

Solução. A função usa um Map em que a chave é o id. Registrar uma turma já existente apenas sobrescreve a mesma chave.

```
// Frontend/src/pages/AcompanhamentosAlunos/AcompanhamentosAlunos.jsx
function opcoesUnicas(registros, campo) {
  const mapa = new Map();
  registros.forEach((registro) => mapa.set(registro[campo].id, registro[campo]));
  return [...mapa.values()];
}

const turmas = opcoesUnicas(registros, 'turma');
const disciplinas = opcoesUnicas(registros, 'disciplina');
```

Ganho. Um único percurso sobre os registros, em O(n), monta cada lista de opções. A mesma função serve para os três filtros.

3.4 Map com chave composta para agrupar o progresso

Problema. O painel inicial mostra o progresso do bimestre por turma e disciplina, como “Turma A · Matemática: 1 de 3 enviados”. Calcular cada grupo filtrando a lista inteira exigiria um percurso completo por grupo.

Solução. A função percorre os registros uma vez e usa um Map cuja chave junta os dois ids, como `"1-3"`. Cada registro encontra o seu grupo em O(1) e incrementa o contador.

```
// Frontend/src/pages/DashboardProfessor/DashboardProfessor.jsx
function montarPendencias(registrosDoBimestre, alunos) {
  const grupos = new Map();

  registrosDoBimestre.forEach((r) => {
    const chave = `${r.turma.id}-${r.disciplina.id}`;
    if (!grupos.has(chave)) {
      grupos.set(chave, {
        chave,
        turma: r.turma,
        disciplina: r.disciplina,
        enviados: 0,
        totalAlunos: alunos.filter((a) => a.idTurma === r.turma.id).length,
      });
    }
    if (JA_ENVIADOS.includes(r.status)) grupos.get(chave).enviados += 1;
  });

  return [...grupos.values()].sort((a, b) =>
    a.turma.nome.localeCompare(b.turma.nome, 'pt-BR'));
}
```

Ganho. O agrupamento e a contagem acontecem em um único percurso, em O(n). O total de alunos da turma é calculado só na primeira vez que o grupo aparece, e não a cada registro.

4. Relação com a Entrega 1: paginação

A paginação estudada na Entrega 1, em C#, foi aplicada na lista de acompanhamentos do professor, que mostra 8 registros por página. O cálculo é o mesmo: a posição inicial da página é (página − 1) × tamanho, e o array é fatiado a partir dela.

```
// Frontend/src/pages/AcompanhamentosAlunos/AcompanhamentosAlunos.jsx
const POR_PAGINA = 8;

const totalPaginas = Math.max(1, Math.ceil(visiveis.length / POR_PAGINA));
const paginaAtual = Math.min(pagina, totalPaginas);
const inicio = (paginaAtual - 1) * POR_PAGINA;
const daPagina = visiveis.slice(inicio, inicio + POR_PAGINA);
```

O `slice` copia apenas os 8 itens da página, e a tabela renderiza só esses itens, independentemente do total. O `Math.min` impede que a página atual fique além da última quando um filtro reduz a lista; ao trocar de filtro, a tela também volta para a página 1.

A diferença em relação à Entrega 1 está na integração: as abas por status, a busca por nome e os filtros reduzem o array antes da paginação, e a contagem “Mostrando 1–8 de 21 registros” vem do tamanho do array já filtrado.

5. Limitações e melhorias futuras

A análise do código mostrou dois pontos que ainda podem ser otimizados com as mesmas estruturas, além da paginação no servidor para volumes maiores.

Lista de senhas comuns em um Set. A validação de senha verifica se a senha está entre 45 senhas comuns com `SENHAS_COMUNS.includes(...)`, em O(n). Trocar o array por um Set e usar `.has(...)` torna a verificação O(1). Com uma lista pequena o ganho prático é mínimo, mas a escolha passa a ser adequada se a lista crescer.

Total de alunos por turma (seção 3.4). O total é calculado com `alunos.filter` uma vez por grupo. Contar os alunos de cada turma em um Map antes do agrupamento evitaria percorrer a lista de alunos várias vezes.

Paginação no servidor. A lista de acompanhamentos é filtrada e paginada no navegador, o que funciona bem para as poucas dezenas de registros de um professor por ano letivo. Para a administração, que vê os registros da escola inteira, o adequado é paginar no banco com `LIMIT` e `OFFSET`, aproveitando os índices criados na Entrega 1 de Banco de Dados, para que apenas a página pedida trafegue pela rede.

6. Conclusão

As estruturas de dados aparecem no ClassSync como escolhas práticas para responder rápido com poucos acessos ao banco. A busca das tags em lote reduziu a listagem de n + 1 para 2 consultas, e o uso de Map e Set trocou percursos aninhados, em O(n²), por percursos únicos, em O(n).

A paginação da Entrega 1 foi levada para a aplicação real e integrada aos filtros da tela. A própria análise apontou melhorias com as mesmas estruturas, como usar um Set na lista de senhas comuns e paginar no banco quando o volume crescer.

Referências

MDN WEB DOCS. Map. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/Map. Acesso em: 3 nov. 2026.

MDN WEB DOCS. Set. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/Set. Acesso em: 3 nov. 2026.

MDN WEB DOCS. Array.prototype.slice(). Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Global_Objects/Array/slice. Acesso em: 3 nov. 2026.

NODE-MYSQL2. Documentação. Disponível em: https://sidorares.github.io/node-mysql2/docs. Acesso em: 3 nov. 2026.